NeuroWork: asistente inteligente para la búsqueda y adaptación laboral de personas neurodivergentes mediante inteligencia artificial generativa y minería de datos

Yahir Ortega Ramos

Instituto Tecnológico de Tijuana, Tijuana, Baja California, México
L21212578@tectijuana.edu.mx

Avance 2 · Repositorio: https://github.com/yahirortegasistemas-bit/-neurowork

Resumen. Este documento reporta el avance del proyecto NeuroWork, una aplicación web (PWA) de apoyo a la búsqueda de empleo para personas neurodivergentes. En esta etapa el sistema pasó de propuesta a un **prototipo funcional** con once módulos operativos, integración de inteligencia artificial generativa mediante un proxy seguro y una **base de datos relacional** (SQLite) conectada a la aplicación. Se incorporó un conjunto de **41 pruebas automáticas** que verifican la analítica, la generación de currículums, la base de datos y la API del servidor. La validación con personas usuarias se planifica para la fase final. El avance global estimado es de aproximadamente 85 %.

Palabras clave: neurodivergencia · búsqueda de empleo · inteligencia artificial generativa · minería de datos · SQLite · accesibilidad.

---

1. Desarrollo funcional parcial

El sistema se encuentra en una **fase operativa inicial estable**. La aplicación contempla once módulos, todos navegables y funcionales:

Módulo	Estado	Función principal
Panel	Funcional	Resumen del día: perfil, checklist, objetivos y dificultad
Progreso	Funcional	Minería de datos y gráficas de avance
Perfil	Funcional	Datos de contacto, experiencia y formación
Habilidades	Funcional	Registro con categoría y nivel de confianza (1–5)
CV	Funcional	Generación/adaptación con IA y cuatro plantillas descargables
Entrevistas	Funcional	Cuatro pestañas: vacante, simulador, preparación y respuestas
Reuniones	Funcional	Instrucciones convertidas en pasos + redacción asistida
Checklist	Funcional	Tareas con repetición diaria
Objetivos	Funcional	Metas con porcentaje de avance

Módulo	Estado	Función principal
Preferencias	Funcional	Modalidad, horario y ajustes razonables
Privacidad	Funcional	Transparencia sobre el manejo de datos

Funcionalidad destacada (CV). Genera un borrador con IA a partir del perfil, permite adaptarlo a una vacante concreta y descargarlo en un documento `.doc` con cuatro diseños profesionales (*Moderno*, *Ejecutivo*, *Clásico* y *Minimalista*), con vista previa antes de descargar y fechas alineadas a la derecha.

Sobre el inicio de sesión. NeuroWork **no implementa autenticación** de forma deliberada. Su modelo es *local-first*: la información permanece en el equipo de la persona y no existe un servidor de cuentas en la nube. Añadir un *login* obligaría a almacenar datos personales en un servidor, lo que contradiría el principio de minimización documentado. Esta decisión puede revisarse si el proyecto requiere operación multiusuario.

2. Prototipo funcional / versión de prueba

El prototipo es **ejecutable y navegable**, e instalable como aplicación (PWA). **Requisitos:** Node.js 22+ y un navegador moderno. **Ejecución:** `node server/server.js` y abrir `http://127.0.0.1:8787`.

Flujo principal demostrable:

1. Registrar perfil y habilidades.
2. Generar un CV con IA y descargarlo en cualquiera de los cuatro diseños.
3. Adaptar el CV a una vacante pegada por la persona.
4. Practicar un simulacro de entrevista con retroalimentación por respuesta.
5. Registrar objetivos y checklist; observar el progreso en gráficas.
6. Comprobar el respaldo en la base de datos (`/api/db`).

La aplicación funciona **sin conexión** para todo lo local mediante un *Service Worker*; solo las funciones de IA requieren red.

3. Avance de la base de datos y arquitectura

3.1 Decisión y estado

Se integró una base de datos **relacional SQLite** usando el módulo nativo `node:sqlite` (sin dependencias externas). El archivo reside en `server/data/neurowork.db`, en el propio equipo, y está excluido del repositorio. **Estado:** diseñada e implementada parcialmente; queda pendiente la migración de datos antiguos y la sincronización multiusuario.

3.2 Esquema (23 tablas con relaciones)

```
workspace (raíz)
 ├─1:1─ profile            ├─1:N─ experiences        ├─1:N─ skills
 ├─1:1─ preferences        ├─1:N─ education          ├─1:N─ languages
 ├─1:1─ prep_plans         ├─1:N─ certifications     ├─1:N─ projects
 ├─1:1─ vacancy_analyses   ├─1:N─ cv_documents ─N:1─ cv_templates
 ├─1:N─ checklist_items    ├─1:N─ goals              ├─1:N─
activity_log
 ├─1:N─ snapshots          ├─1:N─ prepared_answers   ├─1:N─
professional_links
 └─1:N─ meetings ─1:N─ meeting_steps
 └─1:N─ interview_sessions ─1:N─ interview_questions
```

Cada tabla tiene clave primaria; las tablas hijas referencian `workspace(id)` con `ON DELETE CASCADE`. El DDL completo está en `server/db.js`.

3.3 Conexión con la aplicación

Método	Endpoint	Función
PUT	/api/state	Guarda el estado (llamado por `saveState()`)
GET	/api/state	Recupera el estado al arrancar si no había datos locales
DELETE	/api/state	Borra el respaldo («Reiniciar datos»)
GET	/api/db	Motor, ruta y conteo de filas por tabla

La persistencia es doble y transparente: **localStorage siempre** y, si el servidor está activo, **respaldo en SQLite**. Si no hay servidor, la app sigue funcionando igual. La clave de API nunca viaja al navegador: se inyecta en el servidor desde `server/.env`.

4. Evidencias del avance

Las siguientes capturas corresponden al prototipo en ejecución con **datos de ejemplo**, tomadas automáticamente con un navegador en modo sin interfaz.

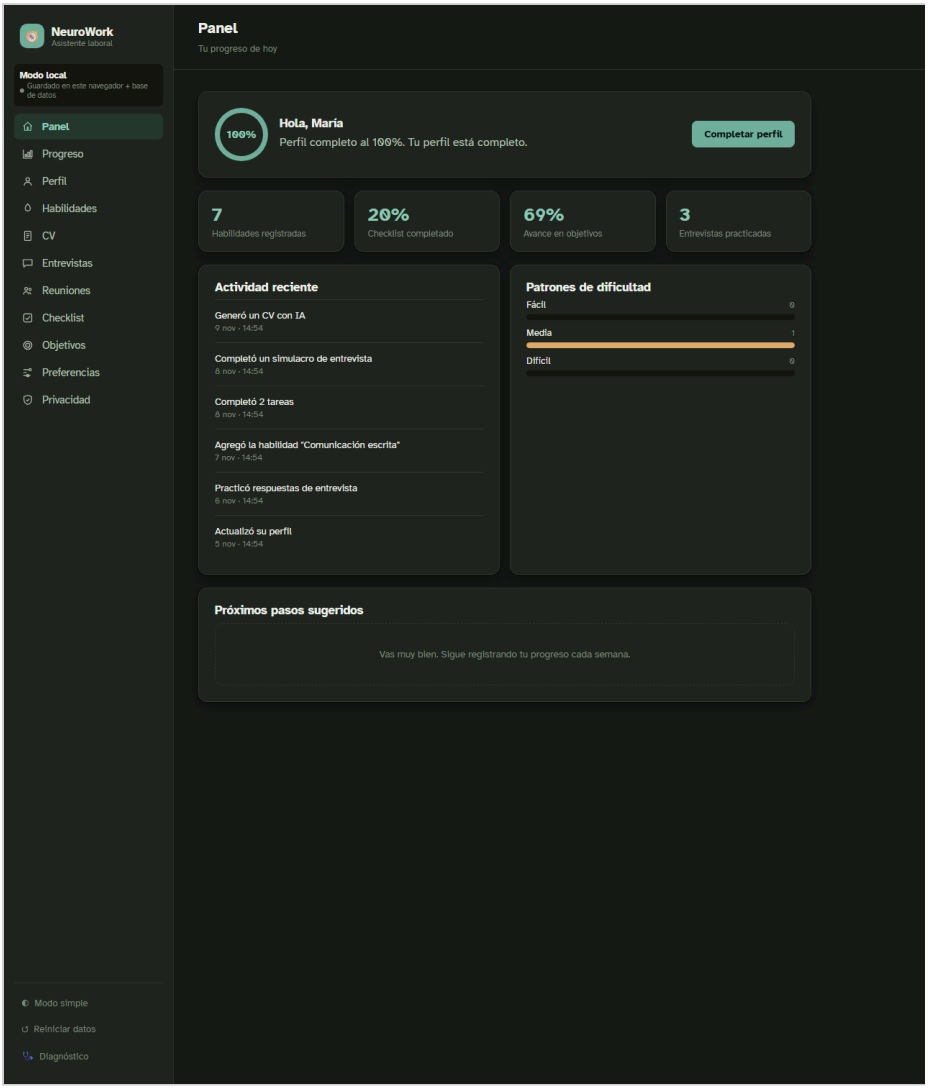


Fig. 1. Panel principal: resumen del progreso del día (perfil, habilidades, checklist, objetivos), actividad reciente y patrones de dificultad.



Fig. 2. Módulo de minería de datos: actividad semanal, evolución del perfil y perfil de habilidades por categoría, calculados a partir de los datos guardados (no se inventan cifras).

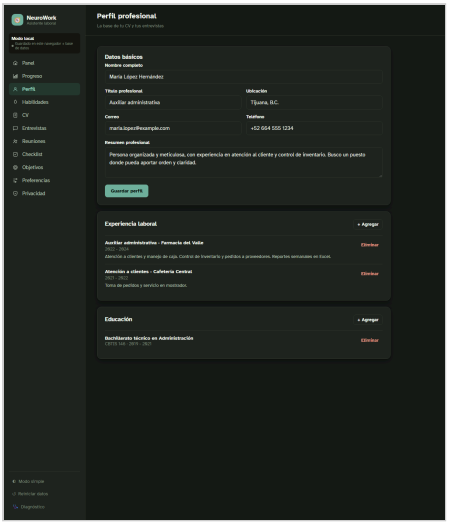


Fig. 3. Perfil profesional: datos de contacto, experiencia y formación; base del CV y de las entrevistas.

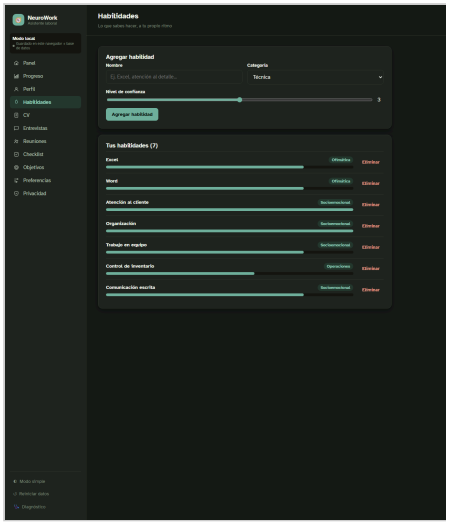


Fig. 4. Registro de habilidades con categoría y nivel de confianza (1–5).

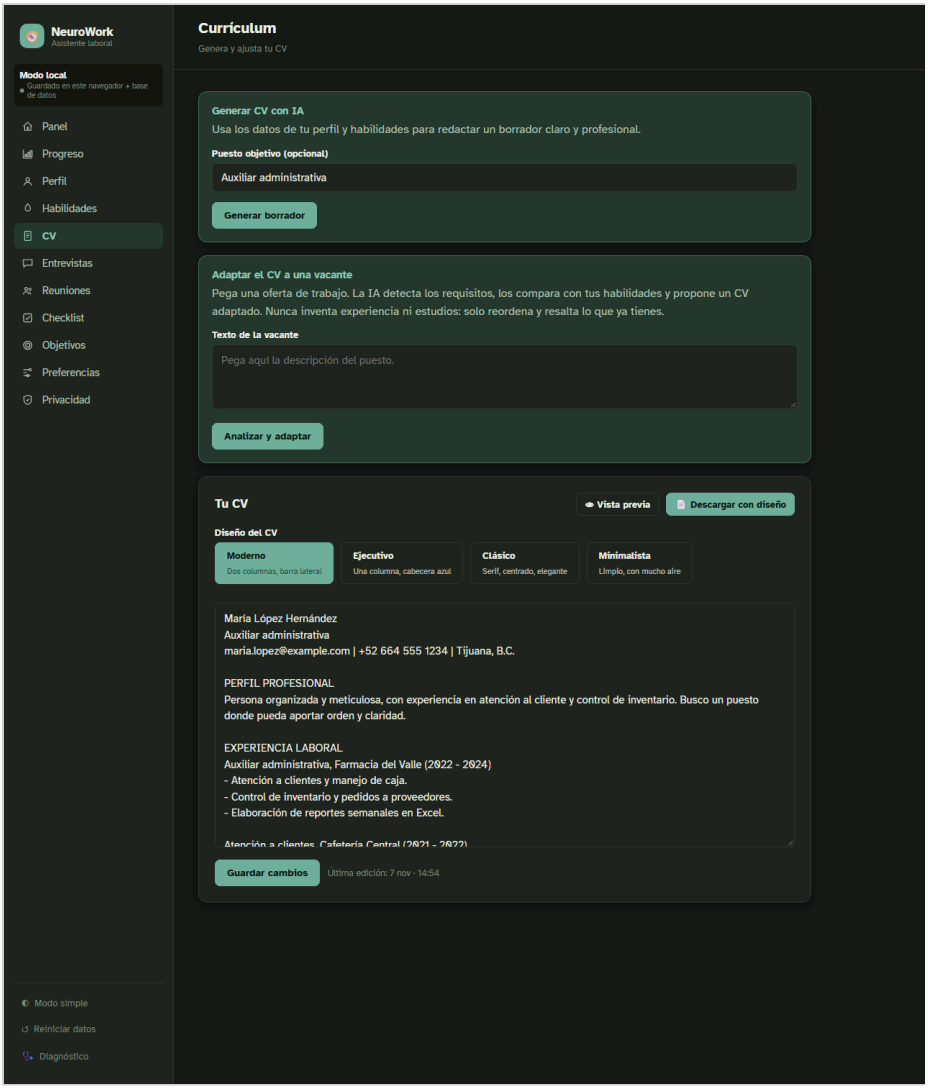


Fig. 5. Módulo de CV: generación asistida por IA, adaptación a una vacante, selector de cuatro diseños, vista previa y descarga.

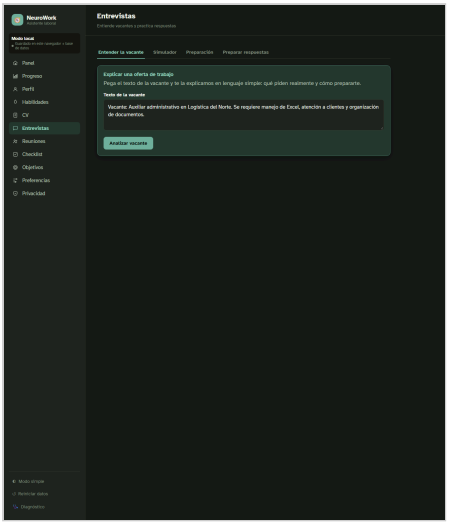


Fig. 6. Entrevistas: cuatro pestañas (entender la vacante, simulador, preparación y preparar respuestas).

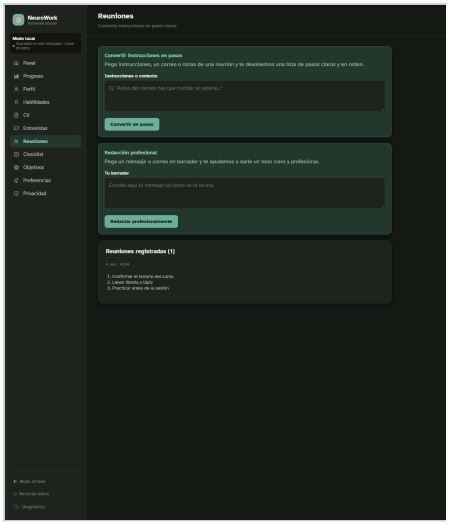


Fig. 7. Reuniones: las instrucciones se convierten en una lista de pasos claros.

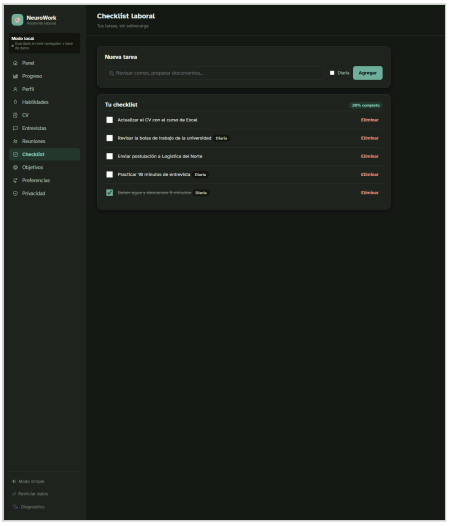


Fig. 8. Checklist de tareas, con opción de repetición diaria.

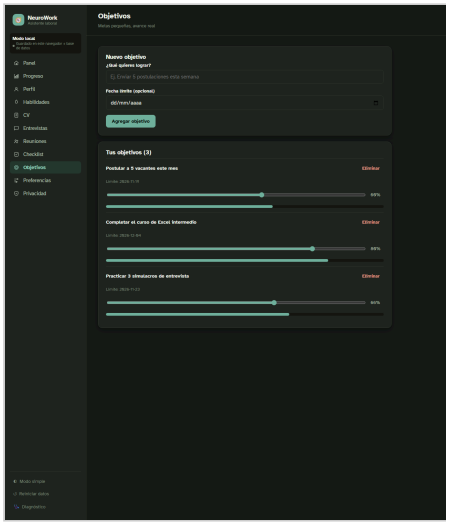


Fig. 9. Objetivos con fecha y porcentaje de avance.

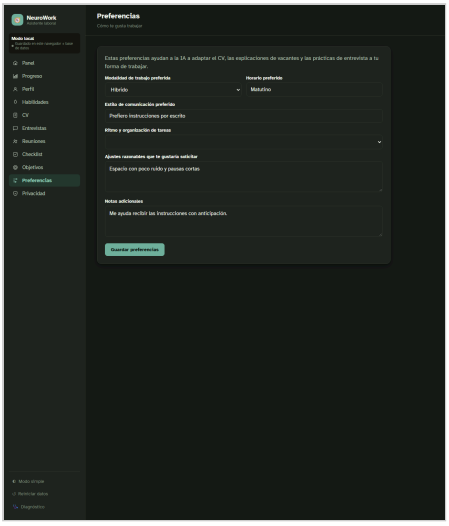


Fig. 10. Preferencias de trabajo: modalidad, horario, comunicación y ajustes razonables.

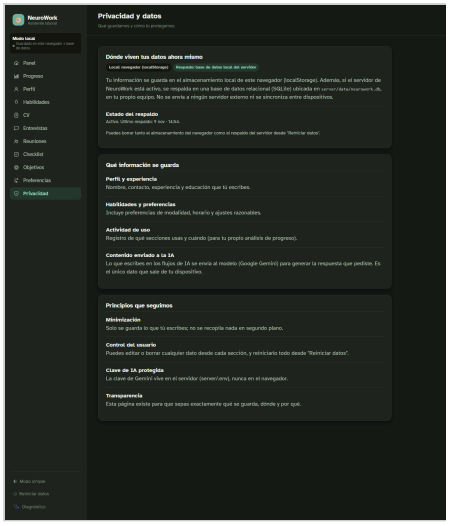


Fig. 11. Privacidad: transparencia sobre los datos y estado del respaldo en la base de datos local.



Fig. 12. Diseño de CV «Moderno»: dos columnas con barra lateral.



Fig. 13. Diseño «Ejecutivo»: una columna con cabecera destacada.

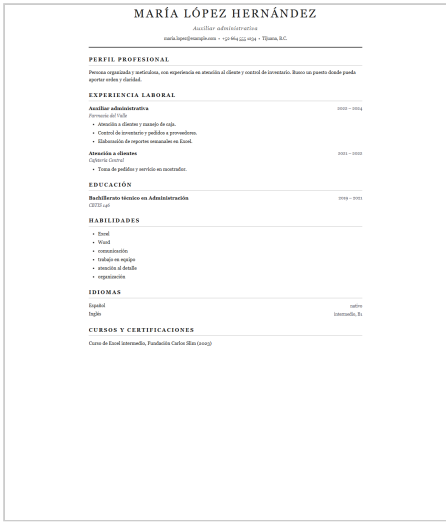
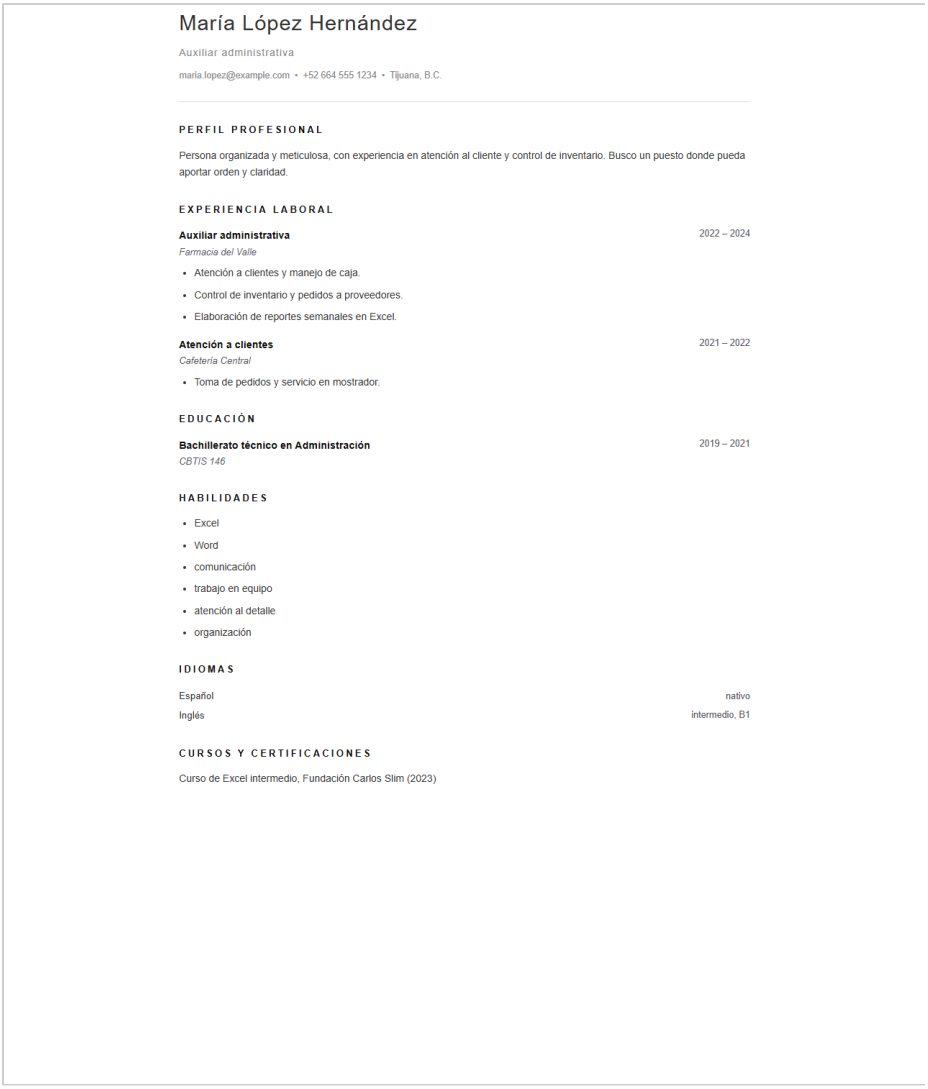


Fig. 14. Diseño «Clásico»: serif y centrado.



María López Hernández

Auxiliar administrativa

maria.lopez@example.com • +52 664 555 1234 • Tijuana, B.C.

PERFIL PROFESIONAL

Persona organizada y meticulosa, con experiencia en atención al cliente y control de inventario. Busco un puesto donde pueda aportar orden y claridad.

EXPERIENCIA LABORAL

Auxiliar administrativa 2022 – 2024
Farmacia del Valle
- Atención a clientes y manejo de caja.
- Control de inventario y pedidos a proveedores.
- Elaboración de reportes semanales en Excel.

Atención a clientes 2021 – 2022
Cafetería Central
- Toma de pedidos y servicio en mostrador.

EDUCACIÓN

Bachillerato técnico en Administración 2019 – 2021
CBTIS 146

HABILIDADES

- Excel
- Word
- comunicación
- trabajo en equipo
- atención al detalle
- organización

IDIOMAS

Español — nativo
Inglés — intermedio, B1

CURSOS Y CERTIFICACIONES

Curso de Excel intermedio, Fundación Carlos Slim (2023)

Fig. 15. Diseño «Minimalista»: limpio y con amplio espacio en blanco.

Comandos de verificación reproducibles: `npm test` (41 pruebas) · `node tools/db-info.js` (23 tablas y conteo de filas) · `curl http://127.0.0.1:8787/api/db`.

5. Comparación con el cronograma inicial

Actividad planificada	Estado	Evidencia
Anteproyecto y análisis de requerimientos	Completado	Descripción general (`docs/01-descripcion-general.md`), anteproyecto

Actividad planificada	Estado	Evidencia
Diseño de arquitectura y modelo de datos	Completado	Arquitectura (`docs/03-arquitectura.md`), Base de datos (`docs/13-base-de-datos.md`)
Diseño de interfaz accesible	Completado	Accesibilidad (`docs/12-accesibilidad.md`), `styles.css`
Desarrollo de módulos funcionales (11)	Completado	app.js
Integración de IA generativa	Completado (técnica)	ai.js, server/server.js
Base de datos relacional	Parcial (local)	server/db.js
Módulo de minería de datos	Completado	Minería de datos (`docs/06-mineria-de-datos.md`), `analytics.js`
Capacidades PWA / offline	Completado	PWA (`docs/08-pwa.md`), `sw.js`
Pruebas técnicas	Completado	`tests/` (41 correctas)
Documentación técnica	En curso	`docs/01–15`
Validación con usuarios	Pendiente	fase final

Avance global estimado: ~85 %. Se consolidaron las actividades técnicas; falta la validación empírica con usuarios y la exportación a PDF.

6. Pruebas preliminares y validaciones

6.1 Pruebas automáticas

Resultado: **41 pruebas, 41 correctas, 0 fallos**.

Archivo	Casos	Cubre
tests/analytics.test.js	16	Minería de datos y gráficas
tests/cv-doc.test.js	7	Generación de documentos de CV
tests/db.test.js	7	Base de datos (ida y vuelta, cascada)
tests/api.test.js	7	API HTTP + base de datos (integración)
tests/server.test.js	4	Seguridad y utilidades del servidor

6.2 Verificación manual

- Verificación sintáctica con `node --check` en `app.js`, `ai.js`, `sw.js` y `server/*.js`.

- `/api/health` operativo con `keyConfigured: true`.
- Comprobación en vivo de `PUT/GET/DELETE /api/state` y `GET /api/db` (23 tablas).
- Lista de comprobación manual en `docs/09-pruebas.md`.

6.3 Correcciones aplicadas

- **IA:** error `Failed to fetch` al abrir la app como archivo; se resolvió sirviendo por HTTP con un mensaje orientador.
- **Caché PWA:** se incrementó la versión del Service Worker para invalidar la caché anterior.
- **CV:** se rediseñó el generador (cuatro plantillas, fechas alineadas, contacto sin duplicados).
- **Codificación:** UTF-8 garantizado en los documentos `.doc` (prefijo BOM).

6.4 Retroalimentación

Aún no se ha realizado validación con usuarios finales. No se reportan opiniones ni resultados que no se hayan recogido formalmente. La prueba piloto con 2–3 personas neurodivergentes está planificada para la fase final.

7. Documentación técnica

- **Diagrama de arquitectura y flujo:** Arquitectura del sistema (`docs/03-arquitectura.md`) y Base de datos (`docs/13-base-de-datos.md`).
- **Especificación de módulos:** Módulos funcionales (`docs/04-modulos-funcionales.md`). **Flujos de IA:** Flujos de IA (`docs/05-flujos-de-ia.md`).
- **Minería de datos:** Minería de datos (`docs/06-mineria-de-datos.md`). **Seguridad:** Seguridad (`docs/07-seguridad.md`).
- **Base de datos:** Base de datos (`docs/13-base-de-datos.md`): ER, tablas, endpoints y mapeo.
- **Manual de usuario inicial:** Manual de usuario (`docs/14-manual-usuario.md`). **Plan de cierre:** Plan de fase final (`docs/15-plan-fase-final.md`).

8. Plan de trabajo para la fase final

#	Actividad pendiente	Entregable	Tiempo estimado
1	Validación con usuarios neurodivergentes (2–3)	Informe de usabilidad	4–5 días
2	Exportación del CV a PDF	Botón «Descargar PDF»	2–3 días

#	Actividad pendiente	Entregable	Tiempo estimado
3	Migración y respaldo de la base de datos	Script de migración	2 días
4	Pruebas E2E completas en navegador	Checklist con evidencias	2 días
5	Revisión de accesibilidad (WCAG 2.2 AA)	Informe de conformidad	2–3 días
6	Refinamiento UI/UX según hallazgos	Ajustes documentados	3 días
7	Documentación final y memoria	Memoria + diagramas	4–5 días
8	Presentación y demo	Diapositivas + video	2 días

Recursos: desarrollo individual, asesoría del profesor y usuarios voluntarios. **Tiempo total estimado:** aproximadamente 4 semanas. **Entregables planeados:** prototipo completo, base de datos con migración, suite de pruebas en verde, memoria técnica, manual de usuario y evidencias (repositorio, capturas, video).